Author: Itssel Sanchez
Date: Apr 15, 2026

Introduction

Music streaming has become an ordinary part of daily life, and platforms like Spotify play a central role in how listeners discover and engage with music. The listening experience today is far more personalized than it was in the past, shaped by recommendations that adapt to individual tastes, habits, and patterns over time. At the core of this transformation is data.

While listener behavior data is not available in this project, Spotify provides extensive information at the track and artist level. This includes audio features that describe how a song sounds, metadata that situates it in time and context, and aggregate measures of engagement such as track popularity. Together, these features offer a partial but meaningful picture of a track and the artist behind it.

Which raises a simple question:

> **Can machine learning predict whether a song is a hit using only what we know about the track and the artist?**

Data

The dataset used in this project was sourced from Kaggle as a single collection containing two files: a tracks dataset and an artists dataset. Together, they provide complementary views of the Spotify catalog, one at the track level and one at the artist level, which were later combined during the data preparation process.

Tracks Dataset

The tracks dataset contains 586,672 tracks spanning from 1921 to 2021. Each row represents a single track. Features can be broadly organized into two groups: audio features, which describe how a song sounds, and track metadata, which situates it in time and context. A popularity score serves as the primary measure of engagement. Tracks are also associated with one or more artists through an artist ID field, which links to the artists dataset. The presence of multi-artist tracks is a notable characteristic of this dataset and required careful handling during data preparation. A full list of feature definitions is provided in Appendix A.

- **Audio Features**: danceability, energy, loudness, speechiness, acousticness, instrumentalness, liveness, valence, tempo, key, mode, time_signature
- **Track Metadata**: release_year, log_duration_min, log_n_artists, explicit

Artists Dataset

The artists dataset contains 1,162,095 artists. Each row represents a single artist. Artists are described by their follower count, popularity score, name, and a genre field containing tags assigned by Spotify. Features can be organized into two groups: artist metrics, which capture reach and engagement, and artist identity, which provides descriptive context. The genre field is sparse, with many artists having no genre tags assigned, which presented a meaningful data quality challenge addressed during data preparation. A full list of feature definitions is provided in Appendix B.

- **Artist Metrics**: followers, popularity
- **Artist Identity**: name, genres

Data Preparation

The raw datasets required several preparation steps before they could be used for modeling. These steps addressed data quality issues, restructured the data to support artist-level enrichment, and produced a clean, analysis-ready dataset.

Cleaning and Standardization

Data types were standardized across both datasets, and the release year was extracted from the raw release date field for use as a modeling feature. List-like fields storing artist IDs and genre tags were parsed from raw strings into structured Python lists to support merging and aggregation.

Duplicate handling was performed in multiple passes. Tracks with identical metadata were identified and consolidated, with popularity values averaged

across duplicates rather than arbitrarily dropped. A final fuzzy matching step caught near-duplicate records that differed only by minor numeric noise, consolidating them into single representative rows. Across all duplicate passes, 9,906 rows were removed.

Missing values were limited to fewer than 0.02% of rows in both datasets and were addressed through complete-case removal with negligible impact on coverage. A small number of records with suspicious zero-value patterns across multiple audio features were also removed as likely data quality artifacts. Tracks released before 1950 were excluded to improve consistency with modern streaming-era popularity patterns, accounting for the majority of rows removed.

Dataset Integration

Because tracks can be associated with multiple artists and artist-level information is stored separately, integrating the two datasets required careful reshaping. Tracks were temporarily expanded into a long track-artist format, enriched with artist metadata, and then aggregated back to one row per track. This approach preserved the one-track-per-row structure required for modeling while adding artist-level context features, including artist popularity, follower counts, genre breadth, and year-relative artist popularity metrics.

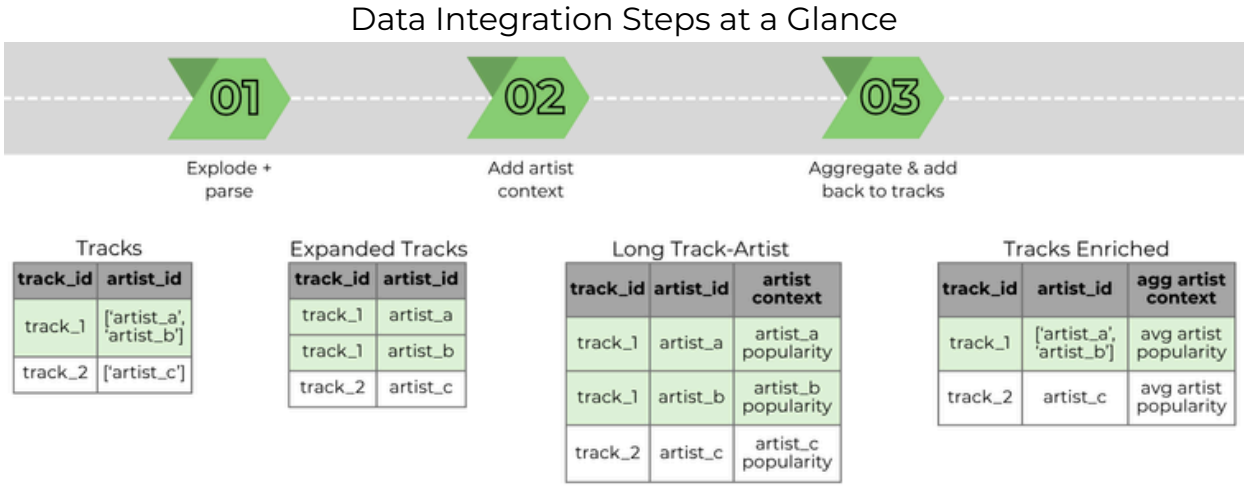

***Figure 1**. Data Integration Steps at a Glance. Data Simplified for illustration. See Appendix for a full list of features and definitions.*

After integration, additional quality filtering removed rows with missing artist context features and records with internally inconsistent popularity and follower signals. Tracks missing genre metadata were retained rather than dropped, with a binary flag added to preserve this information for modeling.

Tracks Enriched Dataset

The resulting enriched dataset contains 528,514 rows and 39 columns, compared to 586,672 rows and 20 columns in the raw tracks table. The reduction in rows reflects targeted quality filtering, while the increase in columns captures the artist-derived context features added through integration. This dataset was carried forward into exploratory analysis where additional feature selection was applied before modeling.

Exploratory Data Analysis

Exploratory analysis examined how features were distributed, how they related to each other, and which signals were most associated with hit status. The goal was to build intuition about the data before modeling and to identify which features were likely to carry a meaningful predictive signal.

What's a Hit?

> **Depends on who you ask.**

For this project, a track was labeled a hit if its popularity score fell in the top 20% of the dataset, corresponding to a cutoff of 45 out of 100. The resulting class distribution is moderately imbalanced, with hits making up approximately 20.5% of the dataset and non-hits making up the remaining 79.5%.

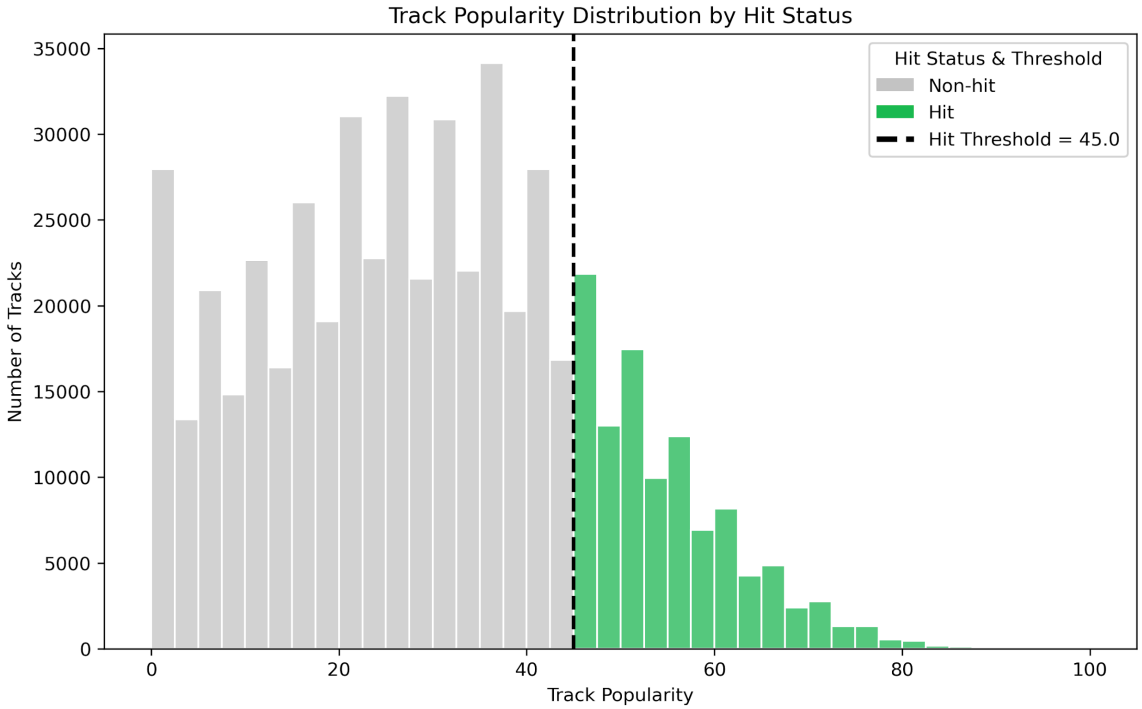

***Figure 2**. Track popularity distribution by hit status. The dashed line indicates the hit threshold at a popularity score of 45.*

What Does the Data Tell Us About Hits?

Exploratory analysis surfaced several patterns that consistently separated hits from non-hits. The findings below highlight the strongest signals, from the most impactful to the most surprising.

Hits tend to be more recent.

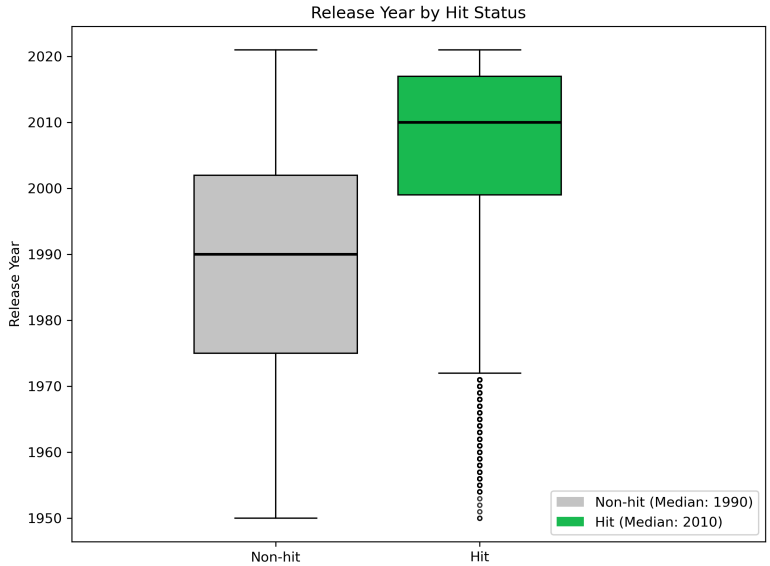

Figure 3. *Release year distribution by hit status. Hit tracks have a median release year of 2010 compared to 1990 for non-hits.*

Bigger artists tend to have bigger hits.

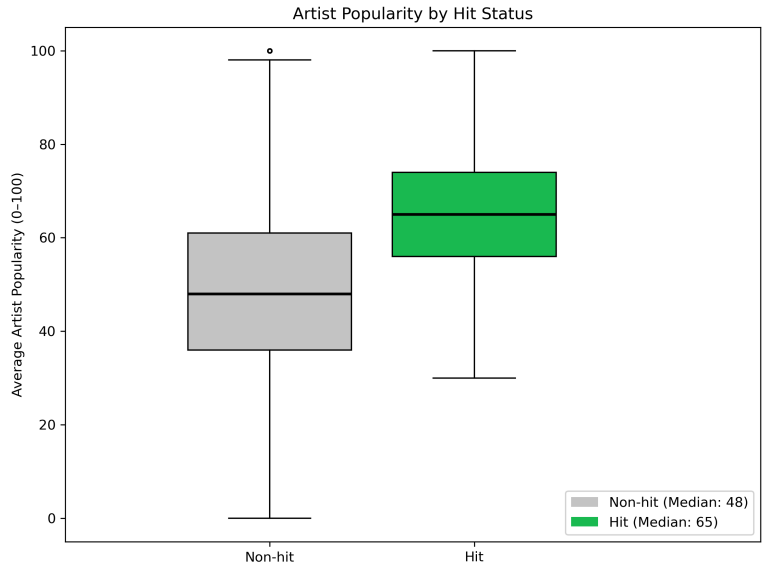

Figure 4. *Artist popularity distribution by hit status. Hit tracks are associated with artists scoring a median popularity of 65 compared to 48 for non-hits*

Hits tend to be louder, more energetic, and danceable than non-hits.

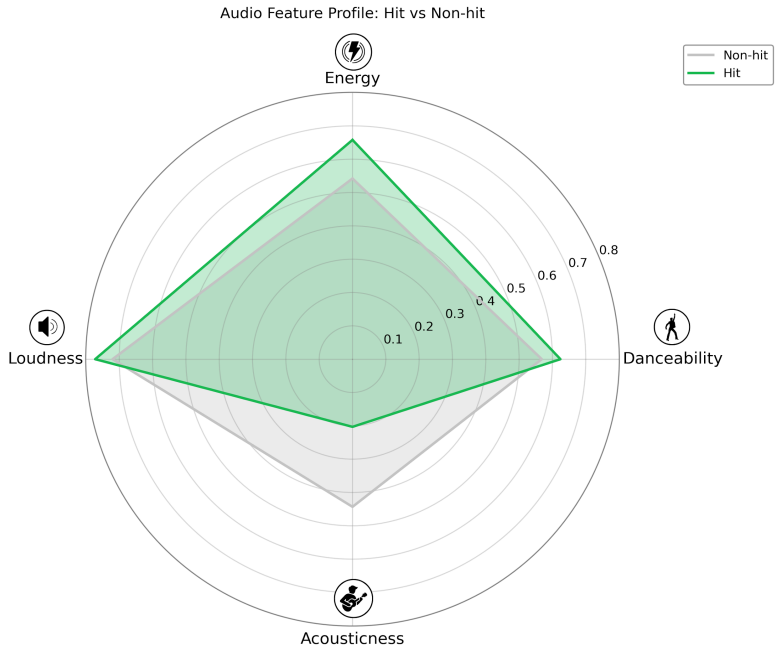

Figure 5. *Audio feature profile comparing hit and non-hit tracks across energy, danceability, loudness, and acousticness.*

Genre matters less than expected.

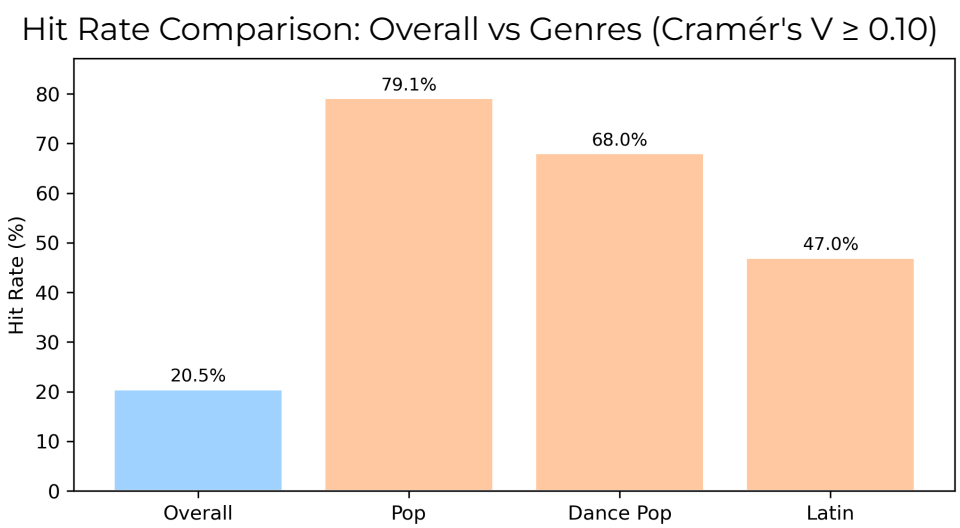

Figure 6. *Hit rate comparison across genres meeting the practical effect threshold (Cramér's V ≥ 0.10).*

Out of all genres analyzed, only three showed a meaningful association with hit status, and even then the effect sizes were modest. Genre is often treated as a defining characteristic of music, yet here it carried far less predictive power than expected. This likely reflects a limitation in how genre is represented, rather than a lack of underlying signal, a question addressed in the modeling section.

Everything was significant. Not everything mattered.

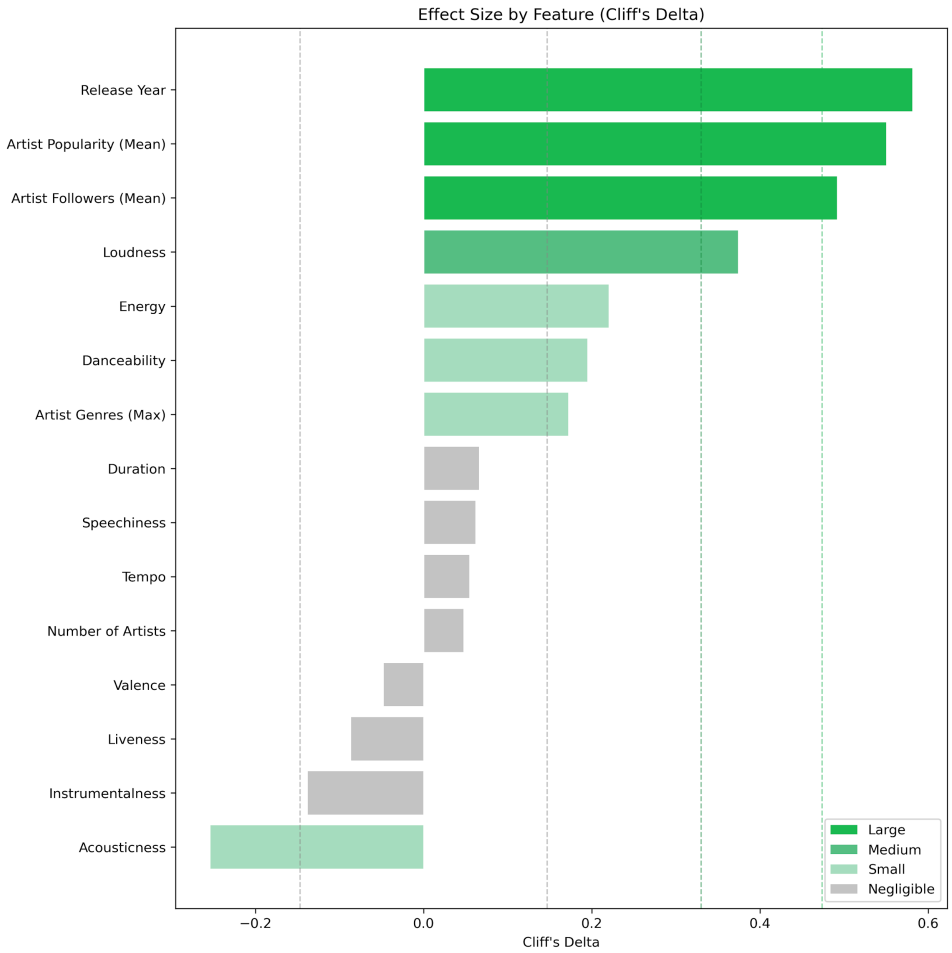

Figure 7. *Effect size by feature using Cliff's Delta. Features are colored by effect size category.*

Effect size analysis using Cliff's Delta confirmed these patterns quantitatively. Release year, artist popularity, and artist followers showed large effect sizes, while most audio features had small to negligible effect sizes. Acousticness was the only audio feature with a meaningful negative association, and Loudness was the only audio feature with a meaningful positive association.

Checking for Redundancy

Before moving to modeling, feature relationships were examined to identify redundancy.

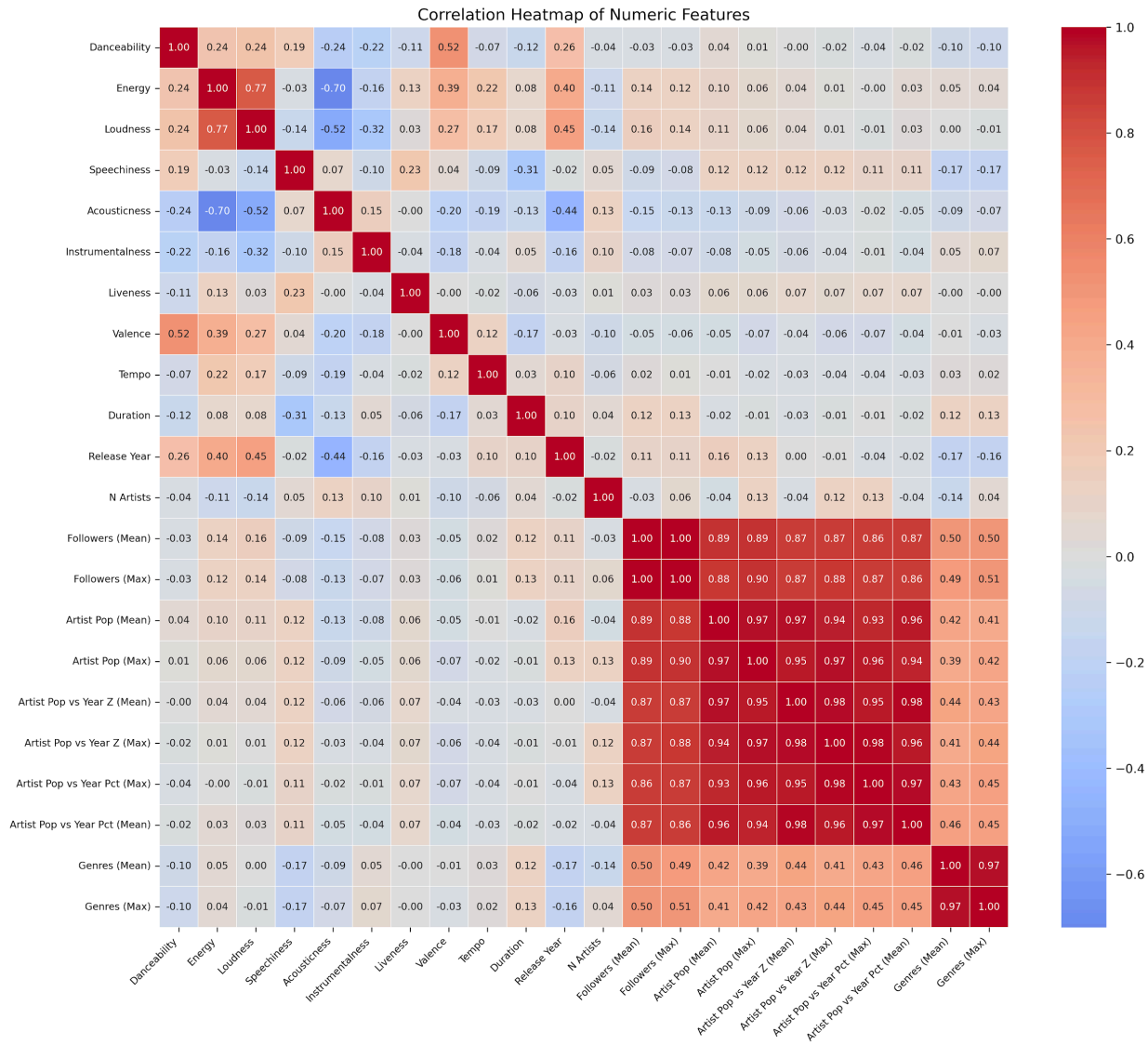

Figure 8. *Correlation heatmap of numeric features. Artist context variables form a tightly correlated cluster in the lower right.*

The correlation heatmap revealed a tightly correlated cluster among artist context features, including popularity, follower counts, and year-relative popularity metrics. This cluster reflects the aggregated artist-level features engineered during data preparation, where multiple versions of the same underlying signal were created. To reduce redundancy, one feature from each highly correlated pair was dropped before modeling, keeping the variant with the stronger association to hit status.

Modeling

The modeling phase evaluates whether hit status can be predicted from track-level audio features, metadata, and artist context, and compares how different model families perform on this structured and semi-structured data.

Feature Engineering: Genre

As observed in exploratory analysis, genre showed a limited predictive signal when treated as categorical labels. However, this likely reflects limitations in representation rather than a lack of underlying signal.

Rather than collapsing genres into broader categories or removing them, genre tags were intentionally preserved in their original form. The genre data is sparse and inconsistently populated, but simplifying it would risk discarding useful information, particularly patterns of genre co-occurrence across tracks.

To address this, genre tags were converted into a single whitespace-separated string per track, treating each tag as a token. TF-IDF was applied to this representation, assigning higher weight to distinctive genres and lower weight to common ones. This approach preserves both granularity and co-occurrence structure, allowing models to learn patterns that traditional categorical encoding cannot capture.

For models requiring dense inputs, the high-dimensional TF-IDF output was compressed using Truncated SVD, reducing it to 300 components while retaining core structure.

Train, Validation, and Test Split

The dataset was split into three stratified subsets: 60% training, 20% validation, and 20% test. The test set was held out from the start and used only for final evaluation. Model selection and threshold tuning were performed on the validation set to prevent information leakage.

Preprocessing Pipelines

Each model family required a different preprocessing approach based on its assumptions and input requirements.

Logistic Regression and MLP require standardized numeric inputs, so numeric features were scaled using StandardScaler. Random Forest and XGBoost are tree-based models that do not require scaling, so numeric features were passed through unchanged. Categorical features were one-hot encoded across all models except TabNet.

Genre was encoded using TF-IDF for all models, with SVD compression applied where dense inputs were required. XGBoost was trained directly on the sparse TF-IDF representation, as it efficiently handles high-dimensional sparse inputs without the need for dimensionality reduction.

TabNet follows a different preprocessing approach. Categorical features were label encoded to provide a compatible integer representation, allowing the model to learn feature embeddings internally. Because TabNet operates outside the scikit-learn pipeline framework, preprocessing was applied directly rather than through a ColumnTransformer.

Preprocessing Pipeline Summary By Model

	Numeric Features	Categorical Features	Genre	Dense Output	Column Transformer
Logistic Regression	Standard Scaler	One-Hot Encoding	TF-IDF	❌	✅
Random Forest	Pass Through	One-Hot Encoding	TF-IDF + SVD	✅	✅
XGBoost	Pass Through	One-Hot Encoding	TF-IDF	❌	✅
MLP	Standard Scaler	One-Hot Encoding	TF-IDF + SVD	✅	✅
TabNet	Standard Scaler	Label Encoding	TF-IDF + SVD	✅	❌

Figure 9. Preprocessing pipeline summary by model. Dense output indicates whether the final feature matrix was converted to a fully dense format before model input.

Models Evaluated

Six models were trained and evaluated, spanning baseline, classical machine learning, and deep learning approaches to compare performance across increasing levels of model complexity. Each model underwent hyperparameter tuning followed by decision threshold tuning on the validation set before final evaluation on the test set.

Baselines	Classic Machine Learning	Deep Learning
Dummy Classifier Trained as a performance floor by predicting the majority class for every track. This establishes a minimum benchmark that all models must exceed to demonstrate meaningful predictive ability.	**Random Forest** An ensemble of independently trained decision trees aggregated through averaging. Captures non-linear relationships and feature interactions without requiring explicit specification.	**MLP** A feedforward neural network with multiple hidden layers, batch normalization, and dropout for regularization. Hyperparameters were tuned using Optuna after initial manual tuning plateaued.
Logistic Regression Serves as a strong linear baseline that captures additive relationships between features. This model provides a reference point for how much predictive signal can be learned without modeling interactions or non-linear effects.	**XGBoost** A gradient boosting model that builds trees sequentially, with each tree correcting the errors of the previous one. Well-suited for structured tabular data with mixed feature types.	**TabNet** A deep learning architecture designed for tabular data that uses a sequential attention mechanism to focus on relevant features at each decision step.

Figure 10. *Models were evaluated across three categories of increasing complexity.*

Evaluation Approach

PR-AUC was used as the primary evaluation metric given the 80/20 class imbalance between non-hits and hits. Unlike ROC-AUC, which can appear overly optimistic in imbalanced settings due to the large number of true negatives, PR-AUC directly reflects the model's ability to prioritize hits over non-hits. A random classifier achieves a score of approximately 0.20 on this dataset, providing a meaningful lower bound for comparison.

Weighted F1 was used as a secondary metric to evaluate classification performance at a selected decision threshold.

Decision threshold tuning was applied after hyperparameter optimization. Instead of defaulting to 0.5, thresholds were swept across a range of values, and the threshold that maximized weighted F1 on the validation set was selected. This improved hit recall across all models with minimal impact on overall performance.

Results

Model Comparison

All five tuned models substantially outperformed the dummy baseline, confirming that meaningful predictive signal exists in the data. PR-AUC ranged from 0.753 for Logistic Regression to 0.811 for Random Forest, compared to the 0.205 random baseline. Weighted F1 ranged from 0.871 to 0.888 across the tuned models, indicating strong overall classification performance across the board.

Model Comparison: Post-Threshold Tuning

Model	PR-AUC	Weighted F1
Random Forest	0.811	0.888
XGBoost	0.809	0.886
MLP	0.801	0.882
TabNet	0.788	0.880
Logistic Regression	0.753	0.871

Figure 11. *Final test set performance for all tuned models, ranked by PR-AUC and Weighted F1.*

The PR-AUC curve provides a more detailed view of model behavior across all classification thresholds. Random Forest and XGBoost tracked closely throughout, sitting at the top of the cluster and remaining nearly indistinguishable across all recall levels. MLP followed closely behind, while TabNet and Logistic Regression showed more visible separation from the tree-based models, particularly in the mid-range recall region.

> **Random Forest and XGBoost are nearly indistinguishable. Tree-based models lead the pack.**

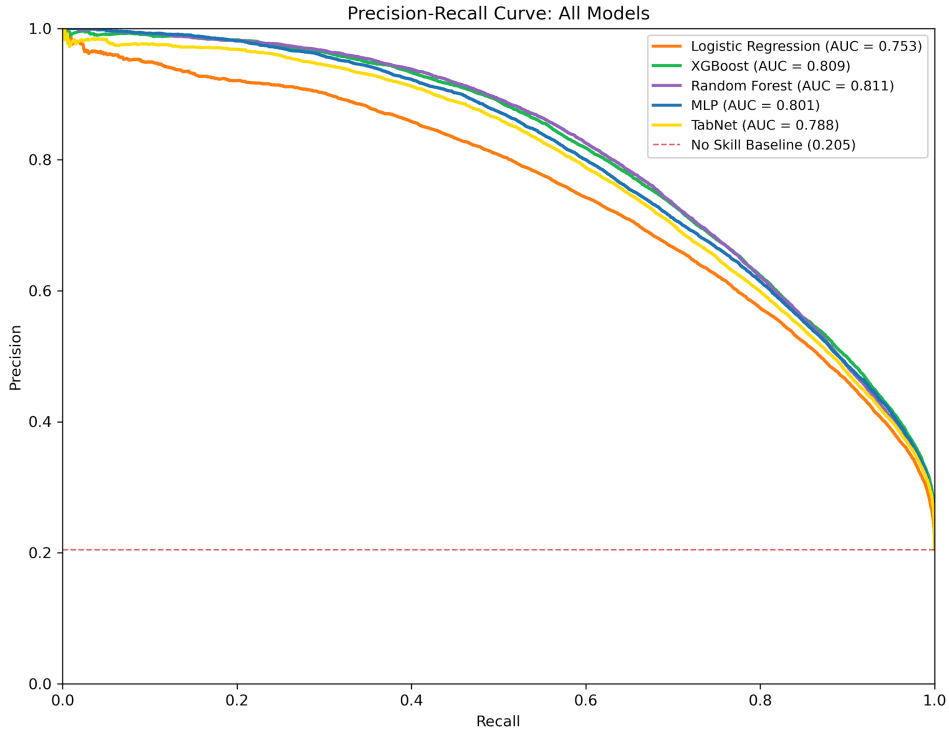

***Figure 12**. Precision-Recall curves for all tuned models. All models remain well above the no-skill baseline of 0.205.*

Tree-based models outperformed both deep learning approaches and Logistic Regression across all metrics, with MLP and TabNet falling behind despite significantly longer training times. Random Forest and XGBoost emerged as the strongest performers, achieving nearly identical PR-AUC scores of 0.811 and 0.809, respectively.

Given this near-identical performance, the selection between the two came down to practical considerations. XGBoost hyperparameter tuning completed in approximately 40 minutes compared to over 9 hours for Random Forest, making it substantially more practical for iterative experimentation and future retraining. XGBoost was therefore selected as the starting point for final tuning. Full per-class classification reports for all models are provided in the appendix.

Final Tuning

With XGBoost selected, a second tuning run was conducted to determine whether additional iterations around the best parameters could improve performance further. The search explored a larger ensemble of deeper trees with a more conservative learning rate, consistent with the general principle that as ensemble size grows, smaller learning steps help prevent overfitting.

V2 finds more hits. V1 is more precise about them.

XGBoost V1 vs V2: Final Tuning Results

Metric	V1	V2	V2 - V1
PR-AUC	0.809	0.812	+0.003
Weighted F1	0.886	0.886	0.000
Non-hit Precision	0.918	0.920	+0.002
Non-hit Recall	0.945	0.941	-0.004
Hit Precision	0.757	0.748	-0.009
Hit Recall	0.671	0.682	+0.011

***Figure 13**. XGBoost V1 vs V2 performance comparison. V1 was evaluated at a threshold of 0.43 and V2 at 0.41.*

PR-AUC improved marginally from 0.809 to 0.812, confirming that additional tuning extracted further signal from the data. The most meaningful change was in hit precision and recall. Hit precision decreased slightly from 0.757 to 0.748, while hit recall improved from 0.671 to 0.682. The lower threshold selected for V2 (0.41 vs 0.43) reflects the model's increased willingness to flag borderline tracks as hits, recovering additional true positives at a modest precision cost.

The final choice between V1 and V2 depended on the intended use. V1 offers slightly higher hit precision while V2 captures more hits overall. For a discovery-oriented context where finding hits matters more than avoiding false alarms, V2 was selected as the final model.

Calibration Curve

Before examining model behavior in detail, probability calibration was assessed to determine whether XGBoost v2's predicted scores can be interpreted as reliable confidence estimates. A well-calibrated model should assign a probability of 0.7 to a track only if approximately 70% of tracks with that score are actual hits. Calibration was evaluated using 10 uniform bins to provide a high-level overview across the full probability range. XGBoost v2 tracks the perfect calibration diagonal closely throughout, with only minor deviations at the low and mid-range probability bins. While finer-grained binning may reveal additional local variation, the overall calibration is strong enough that no post-hoc adjustment such as Platt scaling or isotonic regression is necessary. The scores produced by the model can be interpreted directly as hit confidence estimates.

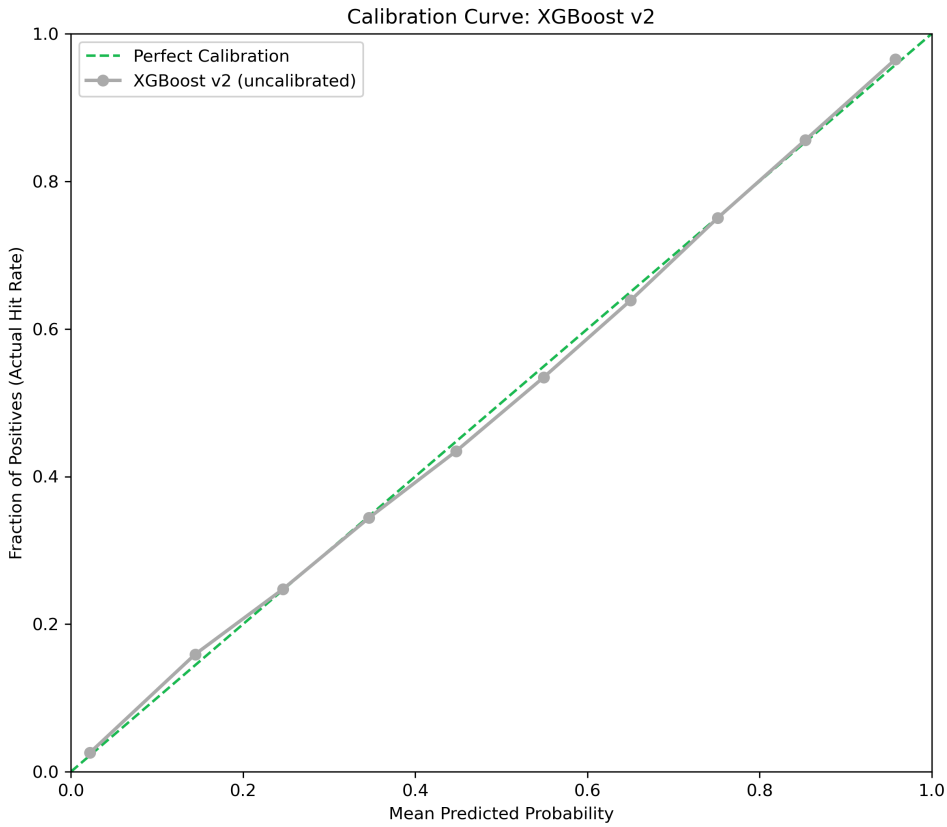

Figure 14. Calibration curve for XGBoost v2 using 10 uniform bins. Points close to the diagonal indicate well-calibrated probability estimates.

Model Interpretation

With XGBoost v2 selected as the final model, this section examines what it has learned and where it struggles. Feature importance analysis identifies which signals drive predictions most strongly, directional analysis reveals how individual features push predictions toward or away from hit status, and error analysis investigates where the model fails and why.

Feature Importance

Permutation importance measures how much PR-AUC decreases when a feature's values are randomly shuffled, breaking its relationship with the target. A large drop indicates the feature carries a meaningful signal.

> **Release year, artist popularity, and genre carry the model. The rest barely register.**

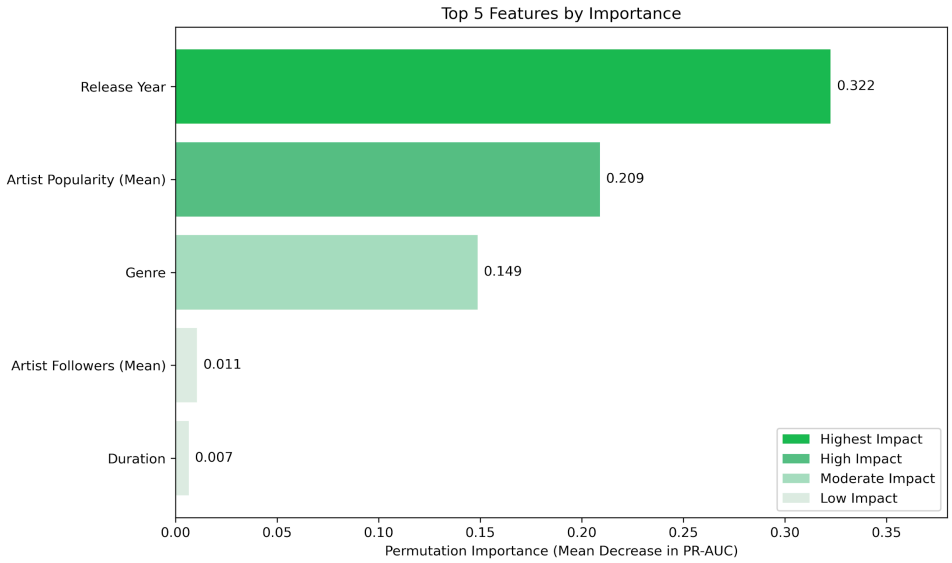

***Figure 15**. Permutation importance for XGBoost v2, ranked by mean PR-AUC decrease across 10 shuffles.*

Release year is by far the most important feature, with a permutation importance of 0.322. Shuffling release year alone causes a drop in PR-AUC nearly twice as large as the next feature. Artist popularity mean follows at 0.209, confirming that an artist's overall standing is a strong predictor of whether an individual track becomes a hit. Genre ranks third at 0.149, indicating that genre context adds meaningful signal beyond release year and artist popularity alone, even when treated as a single aggregated text feature.

There is a dramatic drop after the top three features. Log artist followers falls to 0.011, and all remaining audio features fall below 0.005. Despite being the most intuitive predictors of a song's appeal, audio characteristics like danceability, energy, and tempo contribute relatively little to the model's predictions.

SHAP Analysis

SHAP values provide a more granular view by assigning each feature a contribution to each individual prediction, revealing both the magnitude and direction of each feature's effect.

High artist popularity is no guarantee of a hit. Most audio features have low impacts on individual predictions.

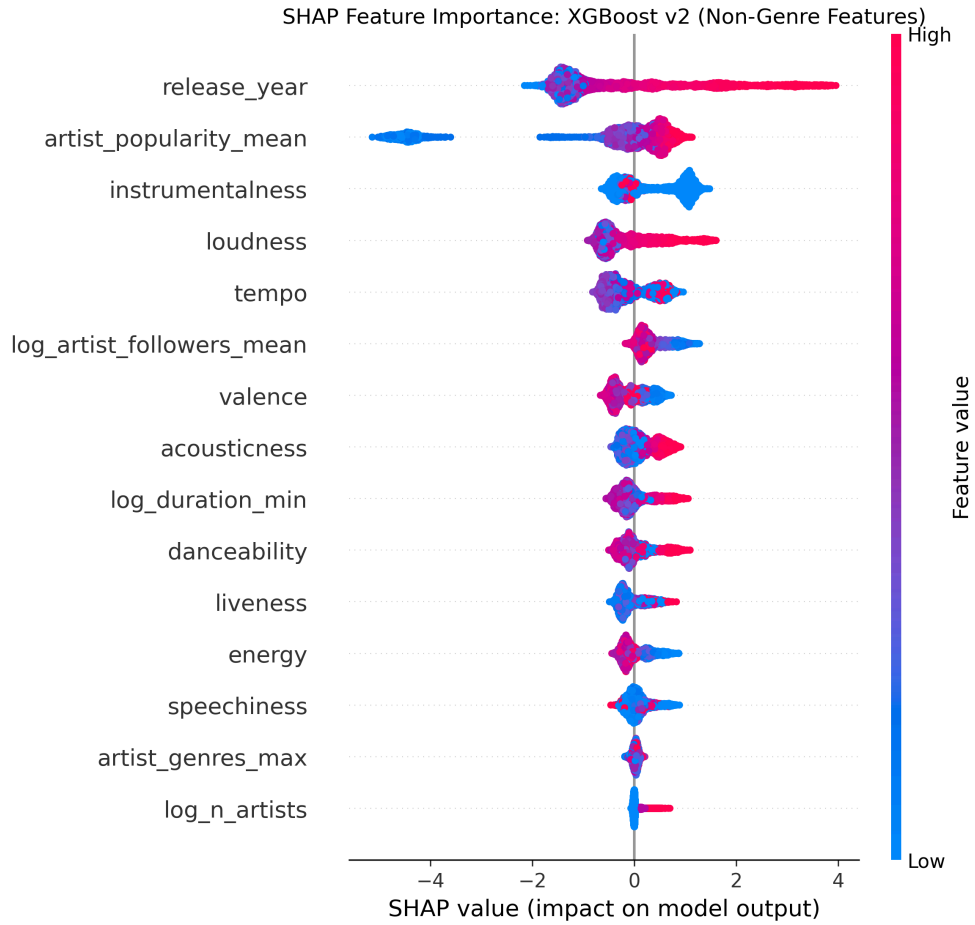

***Figure 16**. SHAP beeswarm plot for XGBoost v2, showing the direction and magnitude of each feature's contribution to individual predictions.*

Release year shows the strongest directional signal. High values push predictions strongly toward hits, while low values push strongly away, reflecting the model's heavy reliance on recency as a proxy for popularity. Artist popularity mean shows an asymmetric pattern: very low values act as a strong disqualifier, while high values create only a modest positive effect. This suggests artist popularity is more useful for ruling out unknowns than for confirming hits.

Loudness is the most directionally meaningful audio feature, with higher values pushing predictions positively, consistent with modern production trends in popular music. Most remaining audio features cluster tightly around zero SHAP values, confirming their limited directional impact on individual predictions.

Error Analysis

Despite strong overall performance, the model misses 31.8% of actual hits. False negatives outnumber false positives (6,870 vs 4,962), indicating the model is more likely to miss a hit than to incorrectly flag a non-hit.

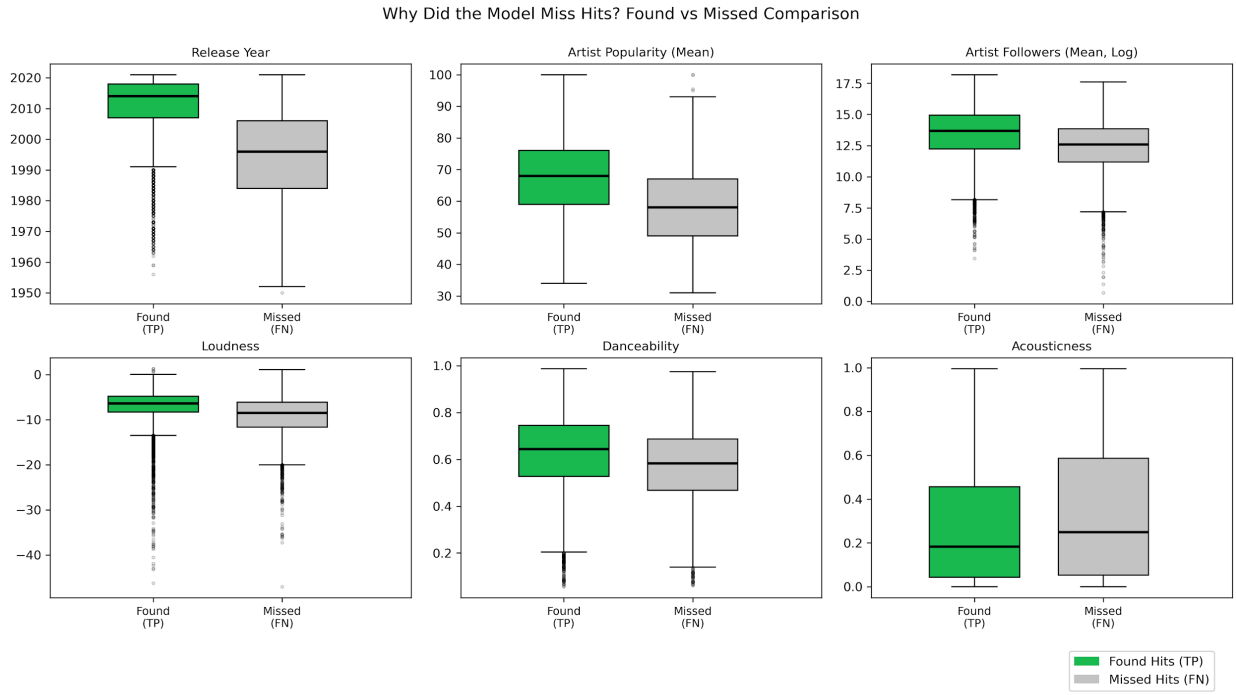

***Figure 17**. Feature distributions comparing correctly identified hits (TP) and missed hits (FN) for XGBoost v2.*

Comparing found and missed hits reveals a systematic pattern. Missed hits are on average 17.5 years older, associated with artists scoring lower on popularity, and tend to be quieter and slightly less danceable and energetic. The model is not failing randomly. It is consistently struggling with older, less prominent artists whose tracks do not match the modern streaming era patterns it was trained on.

The era breakdown makes this quantitative.

The model struggles with older tracks, but excels at modern music.

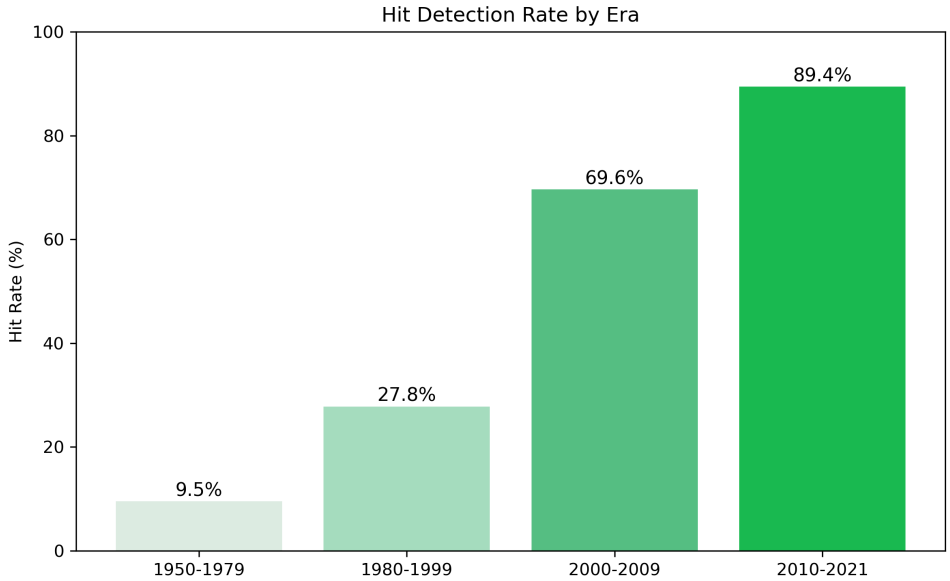

Figure 18. *Hit detection rate by era for XGBoost v2, showing the percentage of actual hits correctly identified within each release year range.*

The model correctly identifies 89.4% of hits from 2010 to 2021, dropping to 69.6% for 2000 to 2009, 27.8% for 1980 to 1999, and just 9.5% for tracks from 1950 to 1979. It is essentially unreliable for pre-1980 music. This is not a modeling failure; it reflects what the model was trained to predict. Older tracks that score as hits in this dataset likely represent songs with sustained or revived streaming relevance, not their original commercial success.

Limitations

The target variable carries inherent limitations. A track is labeled a hit if its popularity score falls in the top 20% of the dataset, an informed but ultimately arbitrary threshold. Beyond the cutoff choice, Spotify's popularity score reflects streaming activity at a snapshot in time rather than historical chart performance, meaning a genuine hit with faded streaming relevance may be labeled a non-hit, while a track with a recent viral moment may be labeled a hit regardless of its original reception. Additionally, Spotify's artist popularity score is partly derived from the popularity of an artist's tracks, introducing potential circularity where artist popularity may partially reflect the very tracks being predicted.

Temporal factors also limit generalizability. Model performance varies significantly by era, correctly identifying 89.4% of hits from 2010 to 2021 but only 9.5% of hits from 1950 to 1979. The dataset also ends in 2021, limiting the

model's ability to generalize to current music trends, evolving listener preferences, and new artists who have emerged since then.

Finally, genre representation remains imperfect. Encoding genre through TF-IDF treats tags as text tokens rather than structured hierarchical categories, which may not fully capture the nuanced relationships between genres and hit potential.

Future Work

The most pressing directions concern how time is handled. Evaluating the model using a temporal train/test split would better reflect real-world prediction, and restricting the dataset to tracks released after 2000 or 2010 would reduce temporal bias and produce a more consistent target variable. Investigating how track popularity changes over time would further help distinguish between timeless hits, era-specific hits, and tracks with revived streaming relevance.

A second set of questions surrounds the target variable itself. Removing or isolating artist popularity features would help assess whether they introduce circularity and measure the true independent contribution of audio and genre signals. Exploring tiered hit classifications, such as mainstream, niche, and viral hits, or using chart appearance data as a more objective ground truth, could address the fundamental limitations of the Spotify popularity score more directly.

Finally, two directions point toward extending the model's reach. Developing features that better capture historical hit status for older tracks, such as chart history data or decade-level popularity normalization, could meaningfully improve performance on pre-2000 music. Deploying the final model as a scoring tool for new track releases, with a focus on recent music where it performs most reliably, represents the most natural next step toward practical application.

Conclusion

This project set out to answer a single question.

> **Can machine learning predict whether a Spotify track is a hit using only track and artist information?**
> **The answer is yes.**

The final XGBoost v2 model achieved a PR-AUC of 0.812 on the held-out test set, well above the 0.205 random baseline. This confirms that meaningful, learnable patterns exist in the data.

More importantly, these patterns reveal something unexpected. The model does not predict hits by learning what makes a song sound good. It predicts hits by learning when a song was released and who made it. Release year and artist popularity dominated every other signal, with audio features contributing relatively little. Popularity, it turns out, is shaped less by the music itself and more by the context surrounding it.

This has real implications for what the model can and cannot do. It performs well on modern music by established artists, correctly identifying nearly 9 in 10 hits from 2010 to 2021. It struggles with older music and less prominent artists, where the signals it relies on are weaker or absent. It is a model of streaming-era popularity, not a model of timeless musical appeal.

Within those boundaries, it works. Understanding those boundaries is as important as the predictions themselves.

Appendices

Appendix A: Tracks Features (586,672 tracks)

Feature	Description
id	Unique Spotify track identifier
name	Track name
popularity	Spotify popularity score at time of data collection (0 to 100)
duration_ms	Track duration in milliseconds
explicit	Whether the track contains explicit content (0 = no, 1 = yes)
artists	List of artist names associated with the track
id_artists	List of Spotify artist IDs associated with the track
release_date	Track release date
danceability	How suitable a track is for dancing based on tempo, rhythm stability, beat strength, and overall regularity (0.0 to 1.0)
energy	Perceptual measure of intensity and activity (0.0 to 1.0)
key	Musical key using standard Pitch Class notation (0 = C, 1 = C#/Db, etc.). -1 if no key detected
loudness	Overall loudness in decibels (dB), typically ranging from -60 to 0 dB
mode	Modality of the track (0 = minor, 1 = major)
speechiness	Presence of spoken words. Above 0.66 = speech-dominant, 0.33–0.66 = mixed, below 0.33 = music (0.0 to 1.0)
acousticness	Confidence measure of whether the track is acoustic (0.0 to 1.0)
instrumentalness	Likelihood that the track contains no vocals. Values above 0.5 indicate likely instrumental (0.0 to 1.0)
liveness	Presence of an audience in the recording. Values above 0.8 indicate likely live performance (0.0 to 1.0)
valence	Musical positiveness conveyed by the track. High = positive, low = negative (0.0 to 1.0)
tempo	Estimated tempo in beats per minute (BPM)
time_signature	Estimated number of beats per measure (range: 3 to 7)

Appendix B: Artists Features (1,162,095 artists)

Feature	Description
id	Unique Spotify artist identifier
followers	Total number of Spotify followers
genres	List of genre tags associated with the artist, as assigned by Spotify
name	Artist name
popularity	Spotify artist popularity score at time of data collection (0 to 100)

Appendix C: Modeling Features (528,514 tracks, 25 features)

***Note**: Features marked with * were engineered and are not present in the raw datasets.*

Feature	Description
release_year*	Year extracted from release date for use as a modeling feature
artist_popularity_mean*	Mean Spotify popularity score across all artists associated with the track (0 to 100)
artist_genres_max*	Maximum number of genre tags across all artists associated with the track
genre_doc*	Deduplicated artist genre tags concatenated into a single whitespace-separated string, used as input to TF-IDF encoding
has_genre_metadata*	Whether genre metadata was available for at least one associated artist (0 = no, 1 = yes)
is_hit*	Target variable. 1 if the track falls in the top 20% of popularity scores, 0 otherwise
log_duration_min*	Log-transformed track duration in minutes
log_n_artists*	Log-transformed number of artists associated with the track
log_artist_followers_mean*	Log-transformed mean follower count across all artists associated with the track
n_artists*	Number of artists associated with the track. Not used as a model input.
artists_key*	Normalized artist name list used for duplicate detection during data preparation. Not used as a model input
track_name_key*	Normalized track name used for duplicate detection during data preparation. Not used as a model input
explicit, danceability energy, key, liveness loudness, mode, tempo speechiness, valence acousticness instrumentalness time_signature	See Appendix A

Appendix D: Classification Reports

Dummy Classifier (Most Frequent Strategy)

	Precision	Recall	F1-Score	Support
Non-hit	0.795	1.000	0.886	84069
Hit	0.000	0.000	0.000	21634
Accuracy			0.795	105703
Macro Avg	0.398	0.500	0.443	105703
Weighted Avg	0.633	0.795	0.705	105703

Logistic Regression (threshold = 0.46)

	Precision	Recall	F1-Score	Support
Non-hit	0.906	0.941	0.923	84069
Hit	0.730	0.619	0.670	21634
Accuracy			0.875	105703
Macro Avg	0.818	0.780	0.797	105703
Weighted Avg	0.870	0.875	0.871	105703

TabNet (threshold = 0.44)

	Precision	Recall	F1-Score	Support
Non-hit	0.913	0.943	0.928	84069
Hit	0.747	0.65	0.695	21634
Accuracy			0.883	105703
Macro Avg	0.830	0.797	0.811	105703
Weighted Avg	0.879	0.883	0.880	105703

MLP (threshold = 0.45)

	Precision	Recall	F1-Score	Support
Non-hit	0.910	0.951	0.930	84069
Hit	0.769	0.635	0.696	21634
Accuracy			0.886	105703
Macro Avg	0.839	0.793	0.813	105703
Weighted Avg	0.881	0.886	0.882	105703

Random Forest (threshold = 0.44)

	Precision	Recall	F1-Score	Support
Non-hit	0.917	0.948	0.932	84069
Hit	0.768	0.667	0.714	21634
Accuracy			0.891	105703
Macro Avg	0.843	0.808	0.823	105703
Weighted Avg	0.887	0.891	0.888	105703

XGBoost V1 (threshold = 0.43)

	Precision	Recall	F1-Score	Support
Non-hit	0.918	0.945	0.931	84069
Hit	0.757	0.671	0.712	21634
Accuracy			0.889	105703
Macro Avg	0.838	0.808	0.821	105703
Weighted Avg	0.885	0.889	0.886	105703

XGBoost V2 (threshold = 0.41)

	Precision	Recall	F1-Score	Support
Non-hit	0.920	0.941	0.930	84069
Hit	0.748	0.682	0.714	21634
Accuracy			0.888	105703
Macro Avg	0.834	0.812	0.822	105703
Weighted Avg	0.885	0.888	0.886	105703